

Coração de Dados — Redes Neurais

Oct 2, 2026 · @JP

Visão geral

Uma rede neural prevê doença cardiovascular com AUC de 0,80 em 13.711 pacientes que ela nunca viu. A ferramenta apoia a **triagem**: indica quem deve ser priorizado para investigação, mas não diagnostica.

Equipe: Adriano, Leonardo e João Paulo.

- **Repositório:** github.com/Nascimentojp/CoracaoDadoRedesNeurais
- **App on-line:** coracaodadoredesneurais.streamlit.app

Em cinco linhas:

1. Base curada de 68.551 pacientes, com 11 dados clínicos simples (idade, pressão, colesterol, hábitos).
2. A mesma rede foi construída em três níveis: do zero em NumPy, em Keras e em PyTorch.
3. As redes empatam com o Gradient Boosting: o teto de AUC ~0,80 vem dos dados, não da arquitetura.
4. O limiar de decisão pesa mais que a arquitetura: com ele a rede encontra 81 % dos doentes, contra 71 % no limiar 0,5.
5. Um app Streamlit e um assistente LangGraph levam o modelo ao uso, com cada número vindo da rede.

Dados e protocolo

A base é o *Cardiovascular Disease dataset* (Kaggle), curada no projeto do Tema 6: 68.551 pacientes, sem duplicatas nem registros fisiologicamente impossíveis. As classes são equilibradas (49,5 % com doença).

Grupo	Variáveis	Tratamento
Contínuas	idade, altura, peso, IMC (derivado), pressão sistólica e diastólica	padronização
Ordinais	colesterol, glicose (1 a 3)	one-hot

Grupo	Variáveis	Tratamento
Binárias	sexo masculino (derivado), fumo, álcool, atividade física	0/1

São 16 entradas para a rede. O protocolo é o mesmo para todos os modelos:

- divisão estratificada 60 % treino, 20 % validação, 20 % teste;
- pré-processador ajustado **somente no treino**;
- parada antecipada e limiar escolhidos na validação;
- teste tocado uma única vez, na avaliação final.

O limiar de triagem é o maior que ainda encontra pelo menos 80 % dos doentes na validação: em triagem, deixar passar um doente custa mais que um alarme falso.

Roteiro técnico

Os 10 notebooks seguem a ordem da disciplina, do neurônio que aprende a soma até um assistente com LLM.

Notebook	O que faz	Conteúdo da disciplina
01 · MLP do zero em NumPy	neurônio da soma, retropropagação à mão, verificação do gradiente, inicialização He, L2	neurônio, ativações, backpropagation
02 · MLP em Keras	Dropout, parada antecipada, class_weight, limiar de triagem	Keras / TensorFlow
03 · MLP em PyTorch	a mesma rede com o laço de treino à vista	PyTorch
04 · Otimizadores e arquiteturas	6 otimizadores, 7 arquiteturas, busca com Keras Tuner (90 redes), agendas da taxa, superfície de decisão	otimização, regularização
05 · Comparação e modelo final	redes × regressão logística × Gradient Boosting, bootstrap pareado, calibração	avaliação de modelos
06 · Explicabilidade	permutação, SHAP, LIME	XAI
07 · Transformer tabular	cada variável vira um token, atenção múltipla, token [CLS]	Transformers, atenção, embeddings

Notebook	O que faz	Conteúdo da disciplina
08 · Autoencoder	mapa latente, comparação com PCA, registros atípicos	redes não supervisionadas
09 · Agente ReAct	LLM que usa a rede como ferramenta	LangChain, agentes, ferramentas
10 · Fluxo LangGraph	Pydantic, caminho explícito, modo demonstração, ReAct × LangGraph	LangGraph, orquestração

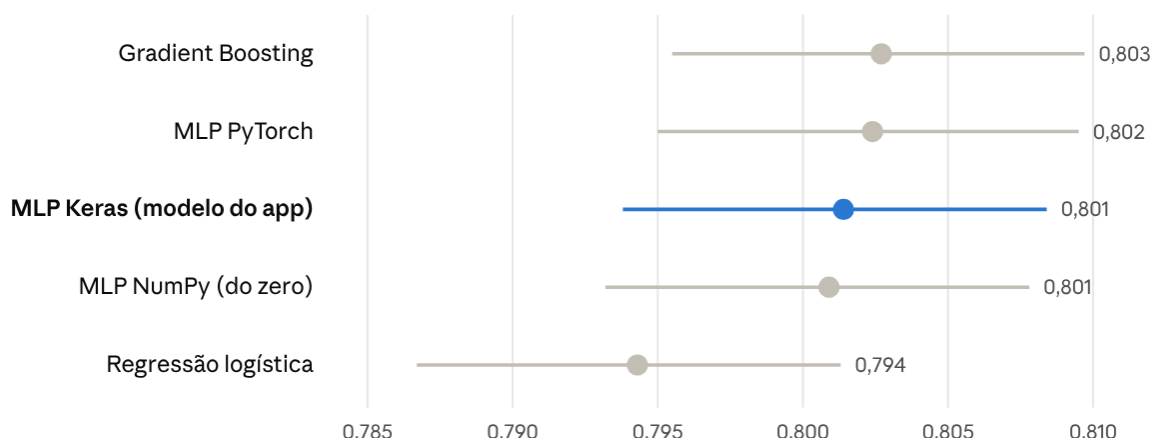
Redes convolucionais, transfer learning e LSTM não foram usados porque a base não tem imagens nem séries temporais (ver Limitações).

Resultados

Todas as redes ficam a no máximo 0,002 de AUC do Gradient Boosting, o melhor modelo clássico para dados tabulares. Redes de 289 a 48 mil parâmetros e a melhor de 90 configurações do Keras Tuner chegam ao mesmo lugar: o teto vem dos dados, não da arquitetura.

Redes e Gradient Boosting empatam em AUC 0,80; a regressão logística fica atrás

AUC no teste (13.711 pacientes) com intervalo de confiança de 95 % por bootstrap



reports/tables/05-comparacao-final/auc-bootstrap.csv · teste, 13.711 pacientes

- **Comparação pareada:** avaliando dois modelos nos mesmos pacientes reamostrados, a regressão logística fica ~0,008 atrás, com diferença significativa em todas as execuções. Entre redes e Gradient Boosting, diferenças de 0,001 às vezes cruzam a significância: estatística sem relevância prática.
- **O limiar pesa mais que a arquitetura** (modelo do app, teste):

Limiar	Doentes encontrados (sensibilidade)	Saudáveis liberados (especificidade)
0,50	71,0 %	76,6 %
0,38 (triagem)	80,8 %	62,9 %

Baixar o limiar encontra 10 pontos a mais de doentes ao custo de mais alarmes falsos, uma decisão de negócio que o app deixa explorar.

Explicabilidade

A pressão sistólica domina a previsão: embaralhá-la derruba a AUC em 0,18, cinco vezes mais que qualquer outra variável. Depois vêm idade e colesterol (0,03 cada); o resto fica abaixo de 0,005. Permutação, SHAP e LIME concordam nessa ordem.

- **Fronteira aprendida sozinha:** entre 35 e 55 anos, a rede cruza $p = 0,5$ em 130–135 mmHg, perto do limiar clínico de hipertensão.
- **Saturação:** acima de ~150 mmHg o risco previsto estabiliza; um único neurônio (reta) não representa isso.
- **Paradoxo de fumo e álcool:** o modelo associa esses hábitos autodeclarados a risco *menor*. É um artefato conhecido desta base, não um efeito protetor, e é o tipo de achado que a explicabilidade existe para expor.
- **Calibração:** as probabilidades erram em média 1 a 2 pontos percentuais, então "risco de 70 %" pode ser lido como frequência real.

Além do MLP: Transformer e autoencoder

Duas arquiteturas dos slides foram aplicadas aos dados reais; nenhuma supera o MLP, e as duas confirmam o que ele aprendeu.

Transformer tabular (notebook 07). Cada variável do paciente vira um token com embedding próprio; 4 cabeças de atenção em 2 blocos; um token [CLS] resume o paciente, como no BERT.

- Com 18 mil parâmetros, empata com o MLP de 3,7 mil (AUC 0,801; a diferença pareada contém zero).
- O [CLS] concentra a atenção na pressão sistólica, seguida de idade e colesterol, a mesma ordem da permutação.

- O padrão exato muda entre execuções: a atenção é uma pista, não uma explicação (Jain & Wallace, 2019).

Autoencoder (notebook 08). A rede comprime as 16 entradas em 2 números sem nunca ver o diagnóstico.

- O mapa latente se abre em dois "braços": pressão alta, quase todos doentes; pressão normal, quase nenhum.
- Reconstrói com 27–38 % menos erro que o PCA, mas não prevê a doença melhor: comprimir bem não é separar bem.
- O erro de reconstrução aponta registros suspeitos, como pressões de 190/60 e 240/100, que passaram pela curadoria; 77 % desse grupo tem a doença, contra 49 % no restante.

Assistente: agente ReAct e fluxo LangGraph

O assistente entende texto livre ("homem, 58 anos, pressão 15 por 9,5"), mas nenhum número de risco sai do LLM: todos vêm da rede neural.

O LLM só extrai e redige; validar e calcular o risco são código



fluxo LangGraph do assistente · 7 nós, src/agente/fluxo.py

Cada resposta mostra o rastreo dos nós percorridos. Os dados se acumulam entre mensagens, então "e se a pressão fosse 12 por 8?" muda só a pressão e recalcula.

- **Pydantic:** o esquema `Paciente` normaliza "1,72 m", "15 por 9,5" e "acima do normal", rejeita valores impossíveis e devolve o erro ao LLM para correção (até 2 tentativas).
- **Modo demonstração:** sem chave de API, regras substituem o LLM; o app público funciona sem custo.
- **Provedores:** OpenAI ou DeepSeek, pelo mesmo adaptador.

Por que um fluxo e não só um agente? No agente ReAct (notebook 09) o LLM escolhe a ordem das ferramentas. Nos testes, ele converteu "15 por 9,5" em 155/95 em 2 de 3 tentativas. No fluxo (notebook 10) a conversão é código e não há como pulá-la. No experimento final, os dois acertaram 3 de 3, e o fluxo usou 2 chamadas ao LLM contra 3: para triagem em saúde, previsibilidade vale mais que flexibilidade.

Aplicação e engenharia

O modelo está publicado num app Streamlit público, e cada mudança passa por 94 testes automáticos antes de chegar a ele.

Aba do app	O que mostra
Paciente	probabilidade, decisão de triagem, curva "e se a pressão fosse outra?", posição entre os 13.711 pacientes de teste, fatores que mais pesaram, mapa pressão × idade
Assistente	conversa em texto livre com o fluxo LangGraph, mostrando cada passo
Desempenho do modelo	limiar interativo com matriz de confusão, calibração, AUC dos modelos com IC, importância das variáveis

Práticas de engenharia:

- **Especificação primeiro:** `specs/01-triagem.md` lista os critérios de aceitação com IDs (VAL, FLX, AGT, APP, MOD), e cada teste cita o critério que verifica.
- **Testes:** 94, cobrindo dados sem vazamento, gradiente, limiar, bootstrap, esquemas, fluxo, agente, app e formato dos números. Nenhum chama API paga. Incluem as armadilhas achadas na revisão: "caso ativo" não é exercício, "fumou por 10 anos" não é idade, "o marido fuma" não é hábito do paciente, "12/08" não é pressão e "não está com colesterol alto" é colesterol normal.
- **Integração contínua:** o GitHub Actions roda lint e testes a cada push.
- **Publicação:** Streamlit Community Cloud, atualizado a cada push na `main`.
- **Formato brasileiro:** números com vírgula decimal (0,801; 80,8%) no app, nos gráficos e nas respostas do assistente.
- **Reprodutibilidade:** ambiente travado com `uv`; figuras e tabelas regravadas por `uv run invoke notebooks`.

Limitações e próximos passos

O limite principal é a base: 11 dados clínicos simples, medidos uma vez, com rótulos ruidosos.

- **Sem imagens nem séries temporais:** por isso redes convolucionais, transfer learning e LSTM não se aplicam aqui. Forçá-las sobre colunas sem ordem seria artificial.
- **Dados pobres:** colesterol e glicose vêm em só 3 níveis; fumo, álcool e atividade são autodeclarados; não há ECG, histórico familiar nem medicação.
- **Faixa de idade:** treinado em adultos de 30 a 65 anos; fora disso a previsão é extrapolação (o app avisa).
- **Não determinismo:** TensorFlow e PyTorch variam na terceira casa decimal entre execuções, sem mudar as conclusões.

Próximos passos possíveis:

1. Com sinais de ECG, aplicar convolução 1D e LSTM, as arquiteturas dos slides que esta base não comporta.
2. Validar o modelo numa base externa antes de qualquer uso real.
3. Trocar a busca por palavras-chave do assistente por RAG com embeddings.

Roteiro da apresentação

Sugestão em três partes, uma por integrante, terminando com a demonstração ao vivo do app.

Parte	Quem	Conteúdo	Mostrar
1. Fundamentos	Adriano	problema, dados e protocolo; rede do zero em NumPy, Keras e PyTorch	notebook 01 (verificação do gradiente)
2. Resultados	Leonardo	comparação com IC, limiar de triagem, calibração, explicabilidade, Transformer e autoencoder	gráfico de AUC; mapa de atenção; mapa do autoencoder
3. Do modelo ao uso	João Paulo	fluxo LangGraph com Pydantic, app, especificação e testes	app ao vivo: aba Paciente, limiar interativo, Assistente

Roteiro da demonstração ao vivo:

1. Aba **Paciente**: mudar a pressão de 130 para 150 e ver a curva "e se" e o mapa responderem.
2. Aba **Desempenho**: arrastar o limiar de 0,38 para 0,5 e ver doentes perdidos subirem.
3. Aba **Assistente**: "Mulher, 61 anos, pressão 14 por 9", completar os dados na mensagem seguinte e perguntar "e se a pressão fosse 12 por 8?".

Antes de apresentar, abra o app alguns minutos antes: no plano gratuito ele adormece após dias sem acesso.

Como executar localmente

```
git clone https://github.com/NascimentoKp/CoracaoDadoRedesNeurais.git
cd CoracaoDadoRedesNeurais
uv sync
cp .env.exemplo .env          # opcional: OPENAI_API_KEY ou DEEPSEEK_API_KEY
uv run invoke app              # app em http://localhost:8501
uv run invoke notebooks       # reexecuta os 10 notebooks (~15 min)
uv run invoke test             # 94 testes
```

Sem chave de API, o assistente roda em modo demonstração, sem custo.